Document de Conception:

Application de réservation de Gîtes

1. Problématique et Solution Minimale

L'offre d'hébergement en domaine rural (gîtes, studios, privatisation complète) est complexe à présenter sur les plateformes classiques. Ceux-ci manquent de spatialisation et obligent à de longs échanges par mail pour trouver un arrangement convenable. La solution minimale proposée est une application web dédiée permettant de visualiser l'agencement du domaine via une carte interactive (HTML/CSS/JS), de consulter les détails de chaque logement, et de soumettre une demande de réservation directement rattachée au compte de l'utilisateur.

On considère l'architecture modèle/vues/contrôleur suivante pour implémenter cette application web:

2. Modèle de données

On utilise une base de données relationnelle SQLite. La gestion des sessions s'appuie sur la ***session*** natif de Flask

a. Tables

- Une table d'utilisateurs ***user*** avec une clé primaire auto-incrémentée ***id***, des colonnes ***email*** et ***password*** (TEXT), et une colonne ***is_admin*** (*booléen*).*
- Une table de logements ***accommodation*** avec une clé primaire auto-incrémentée ***id***, des colonnes ***name***, ***description*** et ***image_url*** contenant du texte, et des colonnes ***capacity*** et ***price*** (INTEGER)
- Une table de réservations ***booking*** avec une clé primaire auto-incrémentée ***id***, deux colonnes ***user_id*** et ***accommodation_id*** (INTEGER), des colonnes ***start_date*** et ***end_date*** (TEXT), et une colonne ***status*** (TEXT : "pending", "confirmed", "refused").

b. Fonctions d'accès

- ***login(email, password)*** : retourne l'id de l'utilisateur si les identifiants sont corrects, -1 sinon.
- ***new_user(email, password)*** : ajoute un utilisateur et retourne son id, ou -1 si l'email existe déjà.
- ***get_accommodations()*** : retourne un tableau de tous les logements disponibles.

**[TRUE / FALSE mais en réalité ce sont des INTEGER en SQLITE]*

- ***get_accommodation(id)*** : retourne les détails d'un logement spécifique.
- ***create_booking(user_id, accommodation_id, start_date, end_date)*** : enregistre une demande dans la table booking.
- ***get_user_bookings(user_id)*** : retourne un tableau des réservations d'un client.
- ***get_all_bookings()*** : retourne toutes les demandes de réservation (pour l'interface administrateur).
- ***update_booking_status(booking_id, status)*** : met à jour l'état d'une demande.

3. Vues

Réalisées grâce aux templates Jinja2.

- Un en-tête ***header*** et un bas de page ***footer*** (qui inclura les informations de contact et d'accès au domaine) utilisés sur toutes les pages, affichant des liens de navigation adaptés selon si l'utilisateur est connecté, déconnecté ou administrateur.
- Une page d'accueil ***index*** *présentant un court paragraphe sur l'histoire du domaine et* affichant un plan visuel du domaine. Client side: un fichier Javascript liée à un fichier CSS permet de rendre la carte interactive (survoler ou cliquer sur un bâtiment highlight ses caractéristiques et affiche un lien vers sa page détaillée).
- Une page de détails ***accommodation*** affichant la description, le prix, et un formulaire permettant de soumettre des dates de réservation (uniquement si l'utilisateur est connecté).
- Des pages ***login*** et ***register*** contenant les formulaires d'authentification et de création de compte.
- Une page ***dashboard*** (pour les clients) listant l'historique de leurs demandes de réservation et leur statut actuel.
- Une page ***admin_dashboard*** (pour les propriétaires) listant toutes les demandes reçues avec des boutons (formulaires) pour les accepter ou les refuser.

4. Routes et logique de contrôle

Réalisées avec Flask.

- ***GET /*** : affiche la page d'index avec la liste et le plan des logements.
- ***GET /accommodation/:id*** : affiche la vue de détails d'un logement spécifique.
- ***GET /login*** et ***GET /register*** : affichent les pages d'authentification.
- ***POST /login*** et ***POST /register*** : gèrent la connexion/création de compte et redirigent vers ***/dashboard*** en cas de succès, ou renvoient la vue avec un message d'erreur.
- ***GET /logout*** : détruit la session et redirige vers ***/***
- ***POST /book*** : accessible uniquement aux utilisateurs connectés. Enregistre la demande de réservation via les données du formulaire et redirige vers ***/dashboard*** avec un message de confirmation.

- ***GET /dashboard*** : accessible aux utilisateurs connectés. Récupère les données via ***get_user_bookings*** et affiche la vue correspondante.
- ***GET /admin*** : accessible uniquement si ***is_admin*** est vrai. Affiche le tableau de bord propriétaire.
- ***POST /admin/update_booking*** : met à jour le statut d'une réservation (acceptée/refusée) et redirige vers ***/admin.***